

hayplot

Plugin de Visualização Declarativa para Hayashi

Grammar of Graphics para a linguagem Hayashi

Baseado em Apache Arrow FFI e Plotters (Rust puro)

Versão hayplot	v0.1.0
Versão Hayashi	0.2.6
Repositório	sheep-farm/hayplot
Licença	MIT

Sumário

1	Introdução	2
1.1	Filosofia	2
1.2	Características Técnicas	2
1.3	Instalação	3
1.4	Importando o Plugin	3
2	Referência das Funções	4
2.1	hayplot(df, aes) — Inicialização	4
2.2	geom_point(plot, color, size) — Camada de Pontos	5
2.3	geom_line(plot, color, size) — Camada de Linha	6
2.4	labs(plot, title, x, y) — Rótulos	6
2.5	render_svg(plot) — Renderização	7
3	Fluxo de Trabalho	9
3.1	Pipeline Declarativo	9
3.2	Tipos de Colunas Suportados	9
3.3	Valores Ausentes	10
4	Exemplos Completos	11
4.1	Dispersão Simples	11
4.2	Curva de Phillips	11
4.3	A partir de Arquivo CSV	12
4.4	Combinando com Regressão OLS	13
4.5	Pipeline Multi-linha	13
4.6	Linha com Marcadores de Pontos	14
5	Arquitetura Técnica	15
5.1	Comunicação via FFI Arrow	15
5.2	Dependências	15
5.3	Compilação e Distribuição	16
6	Guia de Contribuição	17
6.1	Configurando o Ambiente	17
6.2	Testando com o Hayashi Local	17
6.3	Adicionando Novas Geometrias	17
7	Roadmap	19
8	Referência Rápida	20

Capítulo 1

Introdução

hayplot é um plugin nativo para a linguagem Hayashi que implementa uma **Gramática de Gráficos** (*Grammar of Graphics*) ao estilo do **ggplot2** do R. Ele permite construir visualizações de dados de forma declarativa e composicional, usando o operador de pipe `|>` da linguagem.

1.1 Filosofia

A ideia central é que um gráfico pode ser descrito como a composição de *camadas semânticas independentes*:

1. **Dados** — o `DataFrame` de entrada;
2. **Mapeamento estético** (*aes*) — quais colunas mapeiam para x e y ;
3. **Geometrias** (*geoms*) — como os dados serão representados visualmente;
4. **Rótulos** (*labs*) — título e nomes dos eixos.

Cada camada é uma transformação pura de um dicionário de especificação, e a renderização final acontece apenas ao chamar `render_svg()`.

1.2 Características Técnicas

- **Zero-Copy**: os dados do `DataFrame` são compartilhados com o plugin via Apache Arrow FFI sem cópia ou serialização — o ponteiro Arrow é passado diretamente.
- **Rust puro**: usa a crate `plotters` compilada sem dependências externas de sistema (*freetype*, *fontconfig* não são necessários), garantindo portabilidade em Linux, macOS e Windows.
- **Saída SVG**: gera SVG vetorial escalável, adequado para publicação, relatórios em $\text{\LaTeX}$ e inclusão em documentos web.
- **Composicional**: cada função recebe e retorna um `Dict`, tornando qualquer pipeline de plot testável, inspecionável e modificável.

1.3 Instalação

Instale o plugin diretamente do GitHub usando a CLI do Hayashi:

```
hay install sheep-farm/hayplot
```

Listing 1.1: Instalação via CLI

O binário pré-compilado para a sua plataforma é baixado automaticamente da última *GitHub Release*. Para verificar se a instalação ocorreu com sucesso:

```
hay list
```

Listing 1.2: Verificação da instalação

Você deve ver `sheep-farm/hayplot (native so plugin)` na lista de pacotes instalados.

Para atualizar para a versão mais recente:

```
hay update sheep-farm/hayplot
```

Listing 1.3: Atualização

Para verificar a integridade do que está instalado:

```
hay check-plugin sheep-farm/hayplot
```

Listing 1.4: Verificação de integridade

Para remover o plugin:

```
hay remove sheep-farm/hayplot
```

Listing 1.5: Remoção

1.4 Importando o Plugin

Em qualquer script `.hay`, importe o plugin com um alias:

```
1 import("sheep-farm/hayplot", as=gg)
```

Listing 1.6: Importação com alias

A partir daí, todas as funções são acessadas com o prefixo `gg::`.

Capítulo 2

Referência das Funções

hayplot expõe quatro funções públicas. Cada uma recebe e retorna um dicionário de especificação de plot (*plot spec*), tornando a composição via pipe natural e segura.

2.1 hayplot(df, aes) — Inicialização

Inicializa um novo dicionário de especificação de plot com os dados e o mapeamento estético.

Assinatura

```
hayplot(df: DataFrame, aes: Dict) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
df	DataFrame	O DataFrame contendo as colunas a serem plotadas.
aes	Dict	Mapeamento estético. Deve conter as chaves "x" e "y" com os nomes das colunas.

Retorno

Um Dict com a estrutura interna da especificação de plot:

```
{
  "data":    <Arrow DataFrame>,
  "mapping": {"x": "coluna_x", "y": "coluna_y"},
  "layers":  [],
  "labs":    {}
}
```

Exemplo

```

1 let df = dataframe({"pib": [10, 20, 30], "vida": [70, 75, 80]})
2
3 let spec = gg::hayplot(df, {"x": "pib", "y": "vida"})

```

Listing 2.1: Criando a especificação inicial

2.2 geom_point(plot, color, size) — Camada de Pontos

Adiciona uma camada de dispersão (*scatter plot*) à especificação. Cada observação do DataFrame será representada por um círculo.

Assinatura

```
geom_point(plot: Dict, color: String, size: Float) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por hayplot().
color	String	Cor dos pontos. Valores aceitos: "red", "blue", "green", "yellow", "magenta", "cyan", "black". Padrão: "blue".
size	Float	Raio dos pontos em pixels. Valores recomendados: 4.0–12.0.

Retorno

O mesmo Dict da especificação com a camada de pontos adicionada ao campo "layers".

Exemplo

```

1 let spec = gg::hayplot(df, {"x": "pib", "y": "vida"})
2   |> gg::geom_point("red", 8.0)

```

Listing 2.2: Adicionando uma camada de dispersão

Atenção: Nota sobre cores

O nome da cor deve ser passado em inglês e minúsculas. Qualquer cor não reconhecida resultará na cor padrão "blue". O suporte a cores hexadecimais (#RRGGBB) está previsto para versões futuras.

2.3 geom_line(plot, color, size) — Camada de Linha

Adiciona uma camada de linha à especificação, conectando as observações do DataFrame em ordem. Ideal para séries temporais, tendências e gráficos de evolução. Pode ser combinado com `geom_point` para criar gráficos de linhas com marcadores.

Assinatura

```
geom_line(plot: Dict, color: String, size: Float) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .
color	String	Cor da linha. Mesmos valores aceitos por <code>geom_point</code> .
size	Float	Espessura da linha em pixels. Valores recomendados: 1.0–5.0.

Retorno

O mesmo Dict da especificação com a camada de linha adicionada ao campo "layers".

Composição de Camadas

As camadas são renderizadas na **ordem em que são adicionadas**. Para cobrir os pontos sobre a linha (evitando que a linha passe por cima), adicione `geom_line` *antes* de `geom_point`:

```
1 let plot = gg::hayplot(df, {"x": "mes", "y": "vendas"})
2   |> gg::geom_line("blue", 3.0)      // linha primeiro
3   |> gg::geom_point("red", 6.0)     // pontos por cima
4   |> gg::labs("Vendas Mensais", "Mes", "Vendas")
```

Listing 2.3: Gráfico de linha com marcadores de pontos

2.4 labs(plot, title, x, y) — Rótulos

Define o título do gráfico e os rótulos dos eixos horizontal e vertical.

Assinatura

```
labs(plot: Dict, title: String, x: String, y: String) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot.
title	String	Título exibido no topo do gráfico.
x	String	Rótulo do eixo horizontal.
y	String	Rótulo do eixo vertical.

Retorno

O mesmo Dict com o campo "labs" preenchido.

Exemplo

```

1 let spec = gg::hayplot(df, {"x": "pib", "y": "vida"})
2   |> gg::geom_point("blue", 6.0)
3   |> gg::labs("PIB vs. Expectativa de Vida",
4               "PIB per Capita (USD)",
5               "Expectativa de Vida (anos)")

```

Listing 2.4: Definindo título e nomes dos eixos

2.5 render_svg(plot) — Renderização

Compila a especificação de plot e renderiza o gráfico final como uma **string SVG**. Esta é a única função que acessa os dados do DataFrame e executa operações de desenho.

Assinatura

```
render_svg(plot: Dict) -> Result<String, String>
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot completa (com dados, mapeamento, camadas e rótulos).

Retorno

- Em caso de sucesso: uma **String** contendo o código XML do SVG (800×600 pixels).
- Em caso de erro: uma mensagem de erro descritiva como **String**.

Nota: Dimensões do SVG

A saída SVG tem resolução fixa de **800×600 pixels**. Como é um formato vetorial, pode ser redimensionada sem perda de qualidade em qualquer aplicação ou editor de imagens.

Processo interno de renderização

Internamente, `render_svg()` executa os seguintes passos:

1. Extrai o ponteiro Arrow do campo "data" via FFI Zero-Copy;
2. Lê o mapeamento "x" e "y" e recupera as colunas do DataFrame;
3. Calcula os limites automáticos dos eixos com 10% de margem;
4. Itera sobre "layers" e aplica cada geometria;
5. Compila o gráfico para uma string SVG em memória usando `plotters`.

Exemplo

```
1 let svg_content = gg::render_svg(plot)
2 write(svg_content, "saida.svg")
```

Listing 2.5: Renderizando e salvando o SVG

Capítulo 3

Fluxo de Trabalho

3.1 Pipeline Declarativo

O fluxo típico de um gráfico em **hayplot** segue a estrutura abaixo:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // 1. Carregar ou criar os dados
4 load "dados.csv" as df
5
6 // 2. Inicializar a especificacao com dados e mapeamento
7 // 3. Adicionar camadas geometricas
8 // 4. Definir rotulos
9 let plot = gg::hayplot(df, {"x": "col_x", "y": "col_y"})
10      |> gg::geom_point("blue", 6.0)
11      |> gg::labs("Titulo do Grafico", "Eixo X", "Eixo Y")
12
13 // 5. Renderizar e salvar
14 let svg = gg::render_svg(plot)
15 write(svg, "grafico.svg")
```

Listing 3.1: Estrutura geral de um pipeline hayplot

3.2 Tipos de Colunas Suportados

A função `render_svg()` aceita colunas numéricas nos seguintes tipos Arrow:

Tipo Arrow	Tipo Hayashi
Float64	float
Int64	int

Atenção: Colunas categóricas

Colunas do tipo **String** ou booleanas **não** são suportadas como coordenadas x ou y . Para variáveis categóricas, encode-as numericamente antes de plotar.

3.3 Valores Ausentes

Observações com NaN em qualquer uma das coordenadas são automaticamente omitidas da renderização. Não é necessário filtrar os dados antes de plotar.

Capítulo 4

Exemplos Completos

4.1 Dispersão Simples

O exemplo mais básico: dados criados inline, sem arquivos externos.

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // 1. Criar dataset PIB vs Expectativa de Vida
4 let data = {
5     "gdp_per_capita": [12000, 24000, 35000, 48000, 65000,
6                       85000],
7     "life_expectancy": [68.5, 72.1, 76.4, 79.2, 81.5, 83.1]
8 }
9
10 let df = dataframe(data)
11
12 // 2. Construir a especificacao do plot
13 print("Building basic scatter plot...")
14 let plot = gg::hayplot(df, {"x": "gdp_per_capita", "y": "
15     life_expectancy"})
16     |> gg::geom_point("blue", 7.0)
17     |> gg::labs(
18         "GDP vs Life Expectancy",
19         "GDP per Capita (USD)",
20         "Life Expectancy (Years)"
21     )
22
23 // 3. Renderizar e salvar
24 let svg = gg::render_svg(plot)
25 write(svg, "gdp_life_exp.svg")
26 print("Plot saved to 'gdp_life_exp.svg'!")
```

Listing 4.1: examples/basic_scatter.hay

4.2 Curva de Phillips

Clássico da macroeconomia: relação negativa entre desemprego e inflação.

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // 1. Dataset macroeconomico simulado
4 let data = {
5     "unemployment_rate": [3.5, 4.2, 5.0, 6.1, 7.5, 8.8, 9.5],
6     "inflation_rate":    [5.2, 4.5, 3.8, 2.9, 1.8, 1.2, 0.9]
7 }
8 let df = dataframe(data)
9
10 // 2. Construir o grafico da Curva de Phillips
11 print("Building Phillips Curve plot...")
12 let plot = gg::hayplot(df, {"x": "unemployment_rate", "y": "
    inflation_rate"})
13     |> gg::geom_point("red", 8.0)
14     |> gg::labs(
15         "The Phillips Curve",
16         "Unemployment Rate (%)",
17         "Inflation Rate (%)")
18     )
19
20 // 3. Renderizar e salvar
21 let svg = gg::render_svg(plot)
22 write(svg, "phillips_curve.svg")
23 print("Plot saved to 'phillips_curve.svg'!")
```

Listing 4.2: examples/phillips_curve.hay

4.3 A partir de Arquivo CSV

Integrando **hayplot** com o comando **load** do Hayashi:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // 1. Carregar dados externos
4 load "dados_painel.csv" as df
5
6 // 2. Criar grafico
7 let plot = gg::hayplot(df, {"x": "renda", "y": "consumo"})
8     |> gg::geom_point("green", 5.0)
9     |> gg::labs(
10         "Renda vs. Consumo",
11         "Renda (R$)",
12         "Consumo (R$)"
13     )
14
15 // 3. Renderizar
16 let svg = gg::render_svg(plot)
17 write(svg, "renda_consumo.svg")
```

Listing 4.3: Plot a partir de um arquivo externo

4.4 Combinando com Regressão OLS

hayplot integra-se naturalmente ao workflow econométrico do Hayashi. O exemplo a seguir estima um modelo OLS e visualiza os valores ajustados contra os observados:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // 1. Carregar dados e estimar OLS
4 load "painel.csv" as df
5 let m = ols(Y ~ X1 + X2, df)
6
7 // 2. Gerar valores preditos
8 predict df yhat = m
9
10 // 3. Scatter: observado vs predito
11 let plot = gg::hayplot(df, {"x": "yhat", "y": "Y"})
12     |> gg::geom_point("blue", 5.0)
13     |> gg::labs(
14         "Observado vs. Ajustado",
15         "Y-hat (valores preditos)",
16         "Y (valores observados)"
17     )
18
19 // 4. Salvar
20 let svg = gg::render_svg(plot)
21 write(svg, "obs_vs_fit.svg")
```

Listing 4.4: Visualizando valores ajustados de uma regressão OLS

4.5 Pipeline Multi-linha

Com o suporte a quebra de linha no operador `|>` do Hayashi, é possível escrever pipelines mais legíveis:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 let df = dataframe({"x": [1.0, 2.0, 3.0, 4.0, 5.0],
4                     "y": [2.5, 4.1, 5.9, 8.2, 10.0]})
5
6 let svg = gg::hayplot(df, {"x": "x", "y": "y"})
7     |> gg::geom_point("cyan", 9.0)
8     |> gg::labs("Tendencia Linear", "Variavel X", "Variavel Y")
9     |> gg::render_svg
10
11 write(svg, "tendencia.svg")
```

Listing 4.5: Usando quebras de linha na pipeline

4.6 Linha com Marcadores de Pontos

Combinando `geom_line` e `geom_point` em uma única pipeline, é possível criar gráficos de tendência com marcadores visuais em cada observação. A **ordem das camadas** determina qual é desenhada por cima:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // Dados de crescimento de vendas mensais
4 let data = {"month": [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0,
5                 9.0, 10.0, 11.0, 12.0],
6             "sales": [10.5, 12.0, 11.2, 14.8, 16.5, 15.0, 18.2,
7                     21.0, 19.5, 22.8, 25.0, 24.2]}
8 let df = dataframe(data)
9
10 // geom_line adicionado antes de geom_point para que os pontos
11 // fiquem por cima
12 let plot = gg::hayplot(df, {"x": "month", "y": "sales"})
13   |> gg::geom_line("blue", 3.0)
14   |> gg::geom_point("red", 6.0)
15   |> gg::labs("Sales Growth Over Time", "Month", "Sales (
16             Thousands)")
17
18 let svg_content = gg::render_svg(plot)
19 write(svg_content, "sales_growth.svg")
20 print("Plot saved to 'sales_growth.svg'!")
```

Listing 4.6: examples/line_and_scatter.hay

Capítulo 5

Arquitetura Técnica

5.1 Comunicação via FFI Arrow

hayplot é um **plugin nativo** do Hayashi: uma biblioteca dinâmica (`.so/.dll/.dylib`) carregada em tempo de execução. A comunicação entre o interpretador e o plugin segue o protocolo de FFI do **hayashi-plugin-sdk**:

1. O interpretador serializa os argumentos em uma estrutura C-ABI;
2. DataFrames são passados como ponteiros Arrow **ArrayRef**, sem cópia dos dados;
3. O plugin processa em Rust nativo e retorna os valores;
4. O interpretador desserializa o retorno e libera a memória via hooks de desalocação.

Nota: Por que Zero-Copy?

DataFrames econométricos podem ter milhões de linhas. Passar os dados via serialização (JSON, por exemplo) causaria duplicação de memória e *overhead* significativo de CPU. O Arrow FFI resolve isso com um protocolo de ponteiros C padronizado pela Apache Software Foundation.

5.2 Dependências

Crate	Versão	Função
hayashi-plugin-sdk	<i>main</i>	FFI ABI, tipos Arrow, macros de exportação
plotters	0.3.7	Renderização SVG, primitivas gráficas
arrow	(via SDK)	Tipos de dados colunares

plotters é compilado com `default-features = false` e apenas com as features `svg_backend`, `area_series` e `line_series`, eliminando todas as dependências de sistema.

5.3 Compilação e Distribuição

O plugin é compilado como `cdylib` (biblioteca C dinâmica):

```
[lib]
crate-type = ["cdylib"]
```

Listing 5.1: Cargo.toml — tipo de crate

O binário final é distribuído via **GitHub Releases** com CI/CD automático. O Hayashi baixa automaticamente o artefato correto para a arquitetura alvo:

Plataforma	Arquivo
Linux x86_64	hayplot-x86_64-unknown-linux-gnu.so
macOS aarch64	hayplot-aarch64-apple-darwin.dylib
Windows x86_64	hayplot-x86_64-pc-windows-msvc.dll

Capítulo 6

Guia de Contribuição

6.1 Configurando o Ambiente

```
git clone https://github.com/sheep-farm/hayplot.git
cd hayplot
cargo build
```

Listing 6.1: Clonar e compilar localmente

Para compilar em modo de lançamento (otimizado):

```
cargo build --release
```

O arquivo `.so` estará em `target/release/libhayplot.so`.

6.2 Testando com o Hayashi Local

Para usar a versão local do plugin (sem instalar pelo CLI):

```
1 import("./target/debug/libhayplot.so", as=gg)
```

Listing 6.2: Carregando o `.so` compilado localmente

Atenção

O caminho com `./target/debug/` é adequado apenas para desenvolvimento. Em produção, use sempre `import("sheep-farm/hayplot", as=gg)` para carregar a versão instalada via `hay install`.

6.3 Adicionando Novas Geometrias

O padrão para adicionar uma nova geometria (e.g., `geom_line`) é:

1. Criar uma função pública anotada com `#[hayashi_fn]`;
2. A função recebe `mut plot: HashMap<String, HayashiValue>`;
3. Adiciona uma entrada ao vetor `"layers"` com a chave `"geom"`;

4. Retorna o plot modificado.

```
1  #[hayashi_fn]
2  pub fn geom_line(
3      mut plot: HashMap<String, HayashiValue>,
4      color: String,
5      width: f64,
6  ) -> HashMap<String, HayashiValue> {
7      if let Some(HayashiValue::List(ref mut layers)) = plot.
         get_mut("layers") {
8          let mut layer = HashMap::new();
9          layer.insert("geom".to_string(), HayashiValue::Str("
             line".to_string()));
10         layer.insert("color".to_string(), HayashiValue::Str(
             color));
11         layer.insert("width".to_string(), HayashiValue::Float(
             width));
12         layers.push(HayashiValue::Dict(layer));
13     }
14     plot
15 }
```

Listing 6.3: Estrutura de uma nova geometria em Rust

Em seguida, adicione o tratamento correspondente dentro de `render_svg()` no loop de layers.

Capítulo 7

Roadmap

Versão	Feature	Status
v0.1.x	<code>geom_point</code> — dispersão	✓ Concluído
v0.1.x	<code>geom_line</code> — séries temporais e tendência	✓ Concluído
v0.2.0	<code>geom_bar</code> — barras verticais	Planejado
v0.2.0	<code>geom_histogram</code> — histograma	Planejado
v0.3.0	Suporte a cores hexadecimais (<code>#RRGGBB</code>)	Planejado
v0.3.0	<code>scale_x_log10</code> , <code>scale_y_log10</code> — eixos em log	Planejado
v0.3.0	<code>facet_wrap</code> — painéis por grupo	Planejado
v0.4.0	Saída PNG (via feature <code>png_backend</code>)	Futuro
v0.4.0	<code>geom_ribbon</code> — bandas de intervalo de confiança	Futuro
v0.5.0	<code>theme_*</code> — temas de estilo (<code>dark</code> , <code>minimal</code> , <code>classic</code>)	Futuro

Capítulo 8

Referência Rápida

Função	Entrada	Descrição
<code>hayplot(df, aes)</code>	DataFrame, Dict	Inicializa a spec com dados e mapeamento x/y
<code>geom_point(plot, color, size)</code>	Dict, String, Float	Adiciona camada de dispersão (círculos)
<code>geom_line(plot, color, size)</code>	Dict, String, Float	Adiciona camada de linha conectando pontos
<code>labs(plot, title, x, y)</code>	Dict, String, String, String	Define título e nomes dos eixos
<code>render_svg(plot)</code>	Dict	Renderiza e retorna o SVG como String

Cores disponíveis em `geom_point()` e `geom_line()`:

```
"red"      "blue"  "green"  "yellow"
"magenta"  "cyan"   "black"
```

Pipeline mínimo funcional:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 let df = dataframe({"x": [1.0, 2.0, 3.0], "y": [2.0, 4.0, 6.0]})
4 let svg = gg::hayplot(df, {"x": "x", "y": "y"})
5           |> gg::geom_point("blue", 6.0)
6           |> gg::render_svg
7 write(svg, "out.svg")
```

Listing 8.1: Menor script completo